

Introducción a la Programación Orientada a Objetos



En esta clase

- Problemas y soluciones
 - El proceso de desarrollo de **software**
 - Los roles
 - El paradigma de programación Orientada a Objetos
 - Ejemplo: Control de la presión arterial
 - El concepto de **objeto**
 - El concepto de **clase**
- 01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

Problemas y soluciones



Un **programa** es una secuencia de **instrucciones** escritas en un **lenguaje de programación**, que permiten **resolver un problema**.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

Problemas y soluciones



Un **programa** es una componente del **software** que se desarrolla como parte de un **sistema** para **resolver un problema**.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

El proceso de desarrollo de software

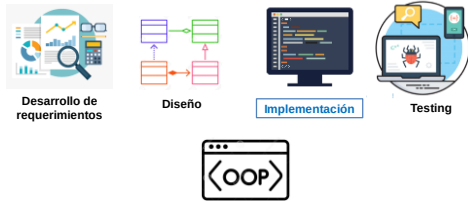


Los roles en el proceso



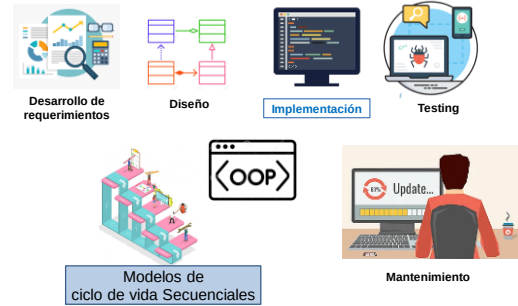
Introducción a la Programación Orientada a Objetos

El paradigma de programación orientada a objetos



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

La gestión de los cambios



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

El proceso de desarrollo de software



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Ejemplo: Control Presión Arterial



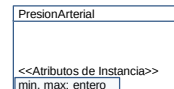
```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Ejemplo: Control Presión Arterial



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Ejemplo: Control Presión Arterial



La **presión arterial** es la fuerza de presión ejercida por la sangre circulante sobre las arterias y constituye uno de los principales signos vitales de un paciente. La presión sistólica se define como el máximo de la curva de presión en las arterias; la presión diastólica es el valor mínimo de la curva de presión. La presión del pulso refleja la diferencia entre las presiones máxima y mínima medidas. Estos valores se miden con un aparato que usa la altura de una columna de mercurio. Si el valor de la presión sistólica o de la presión diastólica está por encima de un cierto umbral, hay un alerta de hipertensión.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Introducción a la Programación Orientada a Objetos

Ejemplo: Control Presión Arterial

```
PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero
```

La presión arterial es la fuerza de presión ejercida por la sangre circulante sobre las arterias y constituye uno de los principales signos vitales de un paciente. **La presión sistólica** se define como el máximo de la curva de presión en las arterias; la **presión diastólica** es el valor mínimo de la curva de presión. La presión del pulso refleja la diferencia entre las presiones máxima y mínima medidas. Estos valores se miden con un aparato que usa la altura de una columna de mercurio. Si el valor de la presión sistólica o de la presión diastólica está por encima de un cierto umbral, hay un alerta de hipertensión.



Ejemplo: Control Presión Arterial

```
PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero

<<Constructor>>
PresionArterial(mi, ma: entero)
```

valores representados en
milímetros de mercurio

PresionArterial(mi, ma: entero)
Requiere $0 < mi < ma$

Ejemplo: Control Presión Arterial

```
PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero

<<Constructor>>
PresionArterial(mi, ma: entero)
<<Consultas>>
obtenerMin(): entero
obtenerMax(): entero
```

valores representados en milímetros de mercurio

```
PresionArterial(mi, ma: entero)
0 < mi < ma
```

Ejemplo: Control Presión Arterial

```
PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero

<<Constructor>>
PresionArterial(mi, ma: entero)

<<Consultas>>
obtenerMin(): entero
obtenerMax(): entero
obtenerPulso(): entero
```

valores representados en milímetros de mercurio

PresionArterial(mi, ma: entero)
 $0 < mi < ma$

obtenerPulso(): entero
max - min

Ejemplo: Control Presión Arterial

PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero
<<Constructor>>
PresionArterial(mi, ma: entero)
<<Consultas>>
obtenerMin(): entero
obtenerMax(): entero
obtenerPulso(): entero
alarmaHipertension(): boolean

valores representados en milímetros de mercurio

```
PresionArterial(mi, ma: entero)
0 < mi < ma
```

obtenerPulso(): entero max - min

```

    alarmaHipertension(): boolean
    Retorna true si
    max > umbralMax o
    min > umbralMin

```

Ejemplo: Control Presión Arterial

PresionArterial

```
class PresionArterial {
```

1	100
0	011
0	110
1	110
1	100
0	011
1	0110
0	1110
1	001
1	11
0	

Creamos una clase usando la palabra reservada **class** seguida del nombre de la clase, el código se delimita con llaves.

Ejemplo: Control Presión Arterial

```
PresionArterial
class PresionArterial {
    /**Valores representados el milimetros de
    mercurio*/
    //Atributos de clase
```

Los comentarios de una línea comienzan con `//`
Si el comentario tiene 2 o más líneas se encierra entre `/*` y `*/`.

Ejemplo: Control Presión Arterial

```
PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140

class PresionArterial {
    /**Valores representados el milimetros de
    mercurio*/
    //Atributos de clase
    private static final int umbralMin = 75;
    private static final int umbralMax = 140;
```

La palabra reservada `static` indica que el valor del atributo es compartido por todos los objetos de la clase `PresionArterial`

La palabra reservada `final` indica que el valor del atributo es constante.

Ejemplo: Control Presión Arterial

```
PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero

class PresionArterial {
    /**Valores representados el milimetros de
    mercurio*/
    //Atributos de clase
    private static final int umbralMin = 75;
    private static final int umbralMax = 140;
    //Atributos de instancia
    private int min;
    private int max;
```

La palabra reservada `private` indica que el alcance de la declaración es la clase completa, un identificador es visible y puede ser accedido dentro de su alcance.

Ejemplo: Control Presión Arterial

```
PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero
<<Constructor>>
PresionArterial(mi, ma: entero)

class PresionArterial {
    /**Valores representados el milimetros de
    mercurio*/
    //Atributos de clase
    private static final int umbralMin = 75;
    private static final int umbralMax = 140;
    //Atributos de instancia
    private int min;
    private int max;
    //Constructor
    public PresionArterial(int mi, int ma){
        //Requiere ma > mi > 0
        max = ma;
        min = mi; }
```

El constructor tiene siempre el mismo nombre que la clase.

Ejemplo: Control Presión Arterial

```
PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero
<<Constructor>>
PresionArterial(mi, ma: entero)

class PresionArterial {
    /**Valores representados el milimetros de
    mercurio*/
    //Atributos de clase
    private static final int umbralMin = 75;
    private static final int umbralMax = 140;
    //Atributos de instancia
    private int min;
    private int max;
    //Constructor
    public
        PresionArterial (int mi, int ma) {
        //Requiere ma > mi > 0
        max = ma; min = mi; }
```

Java es libre de la línea y sensible a mayúsculas y minúsculas.

Ejemplo: Control Presión Arterial

```
PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero
<<Constructor>>
PresionArterial(mi, ma: entero)
<<Consultas>>
obtenerMin(): entero

//Consultas
public int obtenerMin() {
    return min;
}
```

Una **consulta** es un **método** provisto por la clase que no modifica los valores de los atributos y por lo general retorna un resultado.

Ejemplo: Control Presión Arterial

```

PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero

<<Constructor>>
PresionArterial(mi, ma: entero)

<<Consultas>>
obtenerMin(): entero
obtenerMax(): entero
    
```

```

//Consultas
public int obtenerMin() {
    return min;
}
public int obtenerMax() {
    return max;
}
    
```

El **tipo** de la expresión que sigue a la palabra reservada **return** debe ser compatible con el tipo que precede al nombre del método.

Ejemplo: Control Presión Arterial

```

PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero

<<Constructor>>
PresionArterial(mi, ma: entero)

<<Consultas>>
obtenerMin(): entero
obtenerMax(): entero
obtenerPulso(): entero
    
```

```

//Consultas
public int obtenerMin() {
    return min;
}
public int obtenerMax() {
    return max;
}
public int obtenerPulso() {
    //max - min
    return max - min;
}
    
```

El **tipo** de la expresión que sigue a la palabra reservada **return** debe ser compatible con el tipo que precede al nombre del método.

Ejemplo: Control Presión Arterial

```

PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero

<<Constructor>>
PresionArterial(mi, ma: entero)

<<Consultas>>
obtenerMin(): entero
obtenerMax(): entero
obtenerPulso(): entero
alarmaHipertension(): boolean
    
```

```

public boolean alarmaHipertension() {
    /* retorna true si
    max > umbralMax o
    min > umbralMin*/
    return max > umbralMax || min > umbralMin;
}
    
```

El **tipo** de la expresión que sigue a la palabra reservada **return** debe ser compatible con el tipo que precede al nombre del método.

Ejemplo: Control Presión Arterial

```

PresionArterial
<<Atributos de clase>>
umbralMin = 75
umbralMax = 140
<<Atributos de Instancia>>
min, max: entero

<<Constructor>>
PresionArterial(mi, ma: entero)

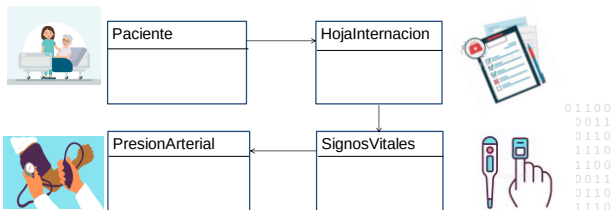
<<Consultas>>
obtenerMin(): entero
obtenerMax(): entero
obtenerPulso(): entero
alarmaHipertension(): boolean
    
```

```

public boolean alarmaHipertension() {
    /* retorna true si
    max > umbralMax o
    min > umbralMin*/
    boolean alarma = false;
    if (max > umbralMax || min > umbralMin)
        alarma = true;
    return alarma;
}
    
```

El **tipo** de la expresión que sigue a la palabra reservada **return** debe ser compatible con el tipo que precede al nombre del método.

Ejemplo: Control Presión Arterial



El **tipo** de la expresión que sigue a la palabra reservada **return** debe ser compatible con el tipo que precede al nombre del método.

Ejemplo: Control Presión Arterial

```

class TesterPresionArterial {
    class PresionArterial {
        /*Valores representados el milímetros de
        mercurio*/
        //Atributos de clase
        private static final int umbralMin = 75;
        private static final int umbralMax = 140;
        //Atributos de instancia
        private int min;
        private int max;
        //Constructor
        public PresionArterial(int mi, int ma){
            //Requiere ma > mi > 0
            max = ma;
            min = mi; }
    }
}
    
```

Antes de integrar este módulo de código al sistema vamos a **verificar** que sus servicios son correctos para un conjunto de casos de prueba.

Ejemplo: Control Presión Arterial

```
class TesterPresionArterial {
    public static void
        main (String a[]) {

class PresionArterial {
    /**Valores representados el milimetros de
    mercurio*/
    //Atributos de clase
    private static final int umbralMin = 75;
    private static final int umbralMax = 140;
    //Atributos de instancia
    private int min;
    private int max;
    //Constructor
    public PresionArterial(int mi, int ma){
    //Requiere ma > mi > 0
    max = ma;
    min = mi; }
```

En Java la ejecución se inicia cuando se ejecuta el método main declarado con la signatura que muestra este ejemplo.

El concepto de clase

Durante el diseño de un sistema una **clase** es un patrón que establece los **atributos** y el **comportamiento** de un **conjunto de objetos**.

En la implementación de un sistema, una **clase** es un **módulo de software** que puede ser desarrollado con alguna independencia del resto de los demás.

El concepto de clase

Nombre
<<Atributos de clase>>
<<Atributos de instancia>>
<<Constructores>>
<<Comandos>>
<<Consultas>>
Responsabilidades

```
class Nombre {
    //Atributos de clase
    //Atributos de instancia
    //Constructores
    //Comandos
    //Consultas
```

Diseño

Implementación

El concepto de objeto

Un **objeto del problema** es una entidad, física o conceptual, que puede ser modelada a través de sus **atributos** y su **comportamiento**.

Un **objeto de software** es un **modelo**, una representación abstracta de un objeto del problema.

En IPOO

En cada problema vamos a

- **implementar** el código en Java de una colección de clases modeladas en un diagrama elaborado en la etapa de **diseño**.
- **verificar** que el código funciona correctamente para un conjunto de casos de prueba.

Para la próxima clase sincrónica

VIDEOS

- Elementos básicos de Java
- Objeto
- Clase
- Diseño de una clase
- Implementación de una clase

ACTIVIDADES PRÁCTICAS

- Elementos básicos de Java
- Implementen en BlueJ o Visual Studio Code